Five Million Users, One Hundred Delegates

Document Number:	P4165R0
Date:	2026-03-28
Audience:	WG21

Reply-to:	Vinnie Falco vinnie.falco@gmail.com Harry Bott haroldjbott@gmail.com

Table of Contents

Abstract

Revision History

- R0: March 2026 (pre-Croydon mailing)

1. Disclosure
2. Two Models
   - 2.1 Model A: Competitive Libraries
   - 2.2 Model B: Standardized Libraries
   - 2.3 What Model B Provides
   - 2.4 Historical Context
3. Economic Foundations
   - 3.1 The Knowledge Problem
   - 3.2 The Calculation Problem
   - 3.3 Creative Destruction
   - 3.4 Collective Action
   - 3.5 Regulatory Capture
   - 3.6 Public Choice
   - 3.7 Self-Interest and Quality
   - 3.8 Summary
4. Predictions
5. Observations
   - 5.1 Observation 1

5.2 Observation 2

5.3 Observation 3

5.4 Observation 4

5.5 Observation 5

5.6 Observation 6

5.7 Observation 7

5.8 Observation 8

5.9 Observation 9

5.10 Observation 10

5.11 Observation 11

5.12 Observation 12

6. The Record

Acknowledgments

References

Abstract

Libraries that must earn their users differ from libraries that are imposed on their users.

Two observable models of library development coexist in C++. One produces libraries through marketplace competition - authors publish code, users adopt or abandon it, and quality is determined by survival. The other produces libraries through committee process - delegates evaluate proposals, vote, and the result ships with every conforming compiler. Economic theory makes specific predictions about the outcomes each model produces. This paper states six predictions derived from established economics, then examines real C++ libraries from both models to test them.

The record is public. The conclusions are the reader's.

Revision History

R0: March 2026 (pre-Croydon mailing)

- Initial version.

---

1. Disclosure

The author provides information and serves at the pleasure of the committee.

The author maintains Boost.Beast[1] and is the founder of the C++ Alliance[2]. The author has a financial interest in the Boost ecosystem. This paper documents two observable models of library development and tests economic predictions against the public record.

The Boost ecosystem competes with the standard library. When this paper examines standard library components alongside Boost alternatives, the author's stake is direct. The reader should weigh every claim in this paper with that bias in mind. Every quotation is attributed. Every document is public. Every fact is independently verifiable.

The author asks for nothing.

---

2. Two Models

Two models of library development are observable in the C++ ecosystem. Both produce libraries. Both serve users. They differ in the mechanisms that connect authors to users, evaluate quality, and determine what survives.

2.1 Model A: Competitive Libraries

Model A libraries are developed in the open marketplace - Boost, GitHub, vcpkg, Conan. The author publishes code. Users evaluate it against alternatives. Adoption is the verdict.

Property	Model A
Author's audience	Users who can choose alternatives
Adoption mechanism	Earned through competition
Feedback signal	Users adopt, switch, or abandon
Lifecycle	Library improves or dies
Cost of entry	Publish code
Cost of defect	Users leave
Knowledge source	Millions of users across all domains

2.2 Model B: Standardized Libraries

Model B libraries are developed through the WG21 process. The author writes a paper. Delegates evaluate it in committee. The result ships with every conforming compiler.

Property	Model B
Author's audience	Voting delegates
Adoption mechanism	Shipped with the compiler
Feedback signal	Committee review, NB comments
Lifecycle	ABI-locked; effectively permanent
Cost of entry	Paper, attendance, years of process
Cost of defect	Workarounds proliferate; defect persists
Knowledge source	~100 delegates at plenary

2.3 What Model B Provides

Model B provides properties that Model A cannot. Portability is guaranteed across every conforming implementation - a `std::optional` on Linux is the same `std::optional` on Windows, on embedded, on every platform the compiler targets. Universal availability eliminates dependency management entirely - no package manager, no build system integration, no version conflicts. Vocabulary type coordination means the entire ecosystem agrees on what `optional`, `variant`, and `string_view` mean, enabling libraries from different authors to compose without adaptation layers. A single authoritative specification provides a reference that courts, contracts, and compliance regimes can cite. These are genuine, structurally important strengths. No marketplace library can replicate them.

2.4 Historical Context

The two models have not always been separate. In 1990, the committee's founding document stated the principle that would govern library standardization: "A key decision was that the Library working group was not in the business of designing new libraries. The key idea is that the Standard would be based on existing practice"[3]. In 1998, Beman Dawes founded Boost[4] to provide peer-reviewed libraries as candidates for standardization. The pipeline from Boost to the standard produced C++11 - the standard's most celebrated release - with `shared_ptr`, `function`, `bind`, `regex`, `random`, `filesystem`, `chrono`, and `thread` all tested through years of Boost deployment before entering the standard[5].

Bjarne Stroustrup observed in his HOPL paper that the committee considered adopting a commercial foundation library in 1992: ‘‘Texas Instruments offered their very nice library for consideration and within an hour five representatives of major corporations made it perfectly clear that if this offer was seriously considered they would propose their own corporate foundation libraries’’[5]. The committee could not adopt one corporation’s library over another. The marketplace - Boost - solved the problem the committee could not.

Joaquín M López Muñoz observed in 2024 that the relationship has changed: ‘‘the standards committee has taken on the role of innovator and is pushing the industry rather than adopting external advancements or coexisting with them’’[6].

---

3. Economic Foundations

The properties described in Section 2 are not unique to software. Economists have studied centralized allocation and competitive markets for centuries. The findings are settled, empirical, and reproduced across domains. This section presents seven results from the economic literature. Each describes an observable property of resource allocation systems. None mentions C++ or WG21.

3.1 The Knowledge Problem

Friedrich Hayek, ‘‘The Use of Knowledge in Society’’ (1945)[7]. No central authority can aggregate the distributed knowledge held by millions of individuals. Each participant holds local knowledge - about their own needs, constraints, and preferences - that is costly or impossible to communicate to a central planner. Markets solve this through price signals that encode the preferences of all participants into a single actionable number. The price aggregates information without requiring any single actor to hold all of it.

3.2 The Calculation Problem

Ludwig von Mises, ‘‘Economic Calculation in the Socialist Commonwealth’’ (1920)[8] and ***Bureaucracy*** (1944)[9]. Even if a central authority could gather dispersed knowledge, without a price signal it cannot calculate which allocation is optimal. Prices emerge from voluntary exchange and encode marginal utility across heterogeneous preferences. Without them, rational resource allocation at scale reduces to guesswork - however well-intentioned the allocators.

In ***Bureaucracy***, Mises distinguishes two management systems. Profit management measures success by an outcome metric - profit or loss - that is external, quantitative, and self-correcting. Bureaucratic management has no equivalent metric. Success is measured by compliance with rules and procedures, because the outcome that the organization exists to produce cannot be priced. The distinction is structural, not a judgment of the people involved.

3.3 Creative Destruction

Joseph Schumpeter, ***Capitalism, Socialism and Democracy*** (1942)[10]. In competitive markets, inferior products are displaced by superior ones. Schumpeter called this process creative destruction - the mechanism through which quality improves over time. New entrants challenge incumbents. Users migrate to the better product. The inferior product loses its user base and either improves or disappears. Where competition is absent or where incumbents are protected from displacement, inferior products persist.

3.4 Collective Action

Mancur Olson, ***The Logic of Collective Action*** (1965)[11]. Small, concentrated groups with strong per-member incentives outperform large, diffuse groups in influencing institutional outcomes - even when the diffuse group's aggregate interest is greater. A firm with three delegates in a standards body has concentrated incentives: the delegates attend every meeting, track every paper, and coordinate their positions. Five million developers who use the standard library have diffuse incentives: no individual developer's stake justifies the cost of participation.

3.5 Regulatory Capture

George Stigler, ''The Theory of Economic Regulation'' (1971)[12]. Nobel Prize in Economics, 1982. Regulatory bodies tend, over time, to serve the interests of the entities with the most representation, rather than the broader public the body was created to serve. The entities that participate most actively in the regulatory process - attending hearings, filing comments, building relationships with regulators - shape the body's output. The broader public, whose interests the body nominally serves, participates less and shapes the output less.

3.6 Public Choice

James Buchanan and Gordon Tullock, ***The Calculus of Consent*** (1962)[13]. Nobel Prize in Economics, 1986. Actors in institutional settings respond to the incentive structures of those institutions, not to abstract public interest. The quality of institutional outcomes depends on the feedback mechanism that connects decisions to consequences. In markets, a bad decision produces a loss. In committees, the feedback mechanism is different: a bad decision produces a paper trail. The distance between the decision and its consequences determines how quickly errors are corrected.

3.7 Self-Interest and Quality

Adam Smith, ***The Wealth of Nations*** (1776)[14]. ''It is not from the benevolence of the butcher, the brewer, or the baker that we expect our dinner, but from their regard to their own interest.'' When self-interest is channeled through competition, the result is quality - the butcher who sells bad meat loses customers to the butcher

across the street. When competition is absent, self-interest produces different outcomes. The insight is not that people are selfish. The insight is that the mechanism - competition or its absence - determines whether self-interest serves the public.

3.8 Summary

Concept	Observable Property	Mechanism
Knowledge problem	Centralized allocators miss distributed needs	Information loss
Calculation problem	Without price signals, allocation reduces to guesswork	Absence of outcome metric
Creative destruction	Inferior products displaced in competitive markets	User choice
Collective action	Concentrated interests outperform diffuse interests	Per-member incentive asymmetry
Regulatory capture	Regulatory output reflects regulated entities	Representation asymmetry
Public choice	Outcome quality tracks feedback mechanism quality	Decision-consequence coupling
Self-interest	Competition channels self-interest toward quality	Rivalry for adoption

4. Predictions

If the economic findings in Section 3 apply to software library development, then the two models described in Section 2 should produce different observable outcomes. This section derives six predictions. Each states what we should expect to observe under Model A and Model B. The predictions are numbered for reference.

Prediction 1 (Hayek). Libraries developed under Model A should more accurately reflect the needs of the broader user community than libraries developed under Model B, because Model A aggregates information from a larger population through adoption signals.

Prediction 2 (Mises). Model B's evaluation process should substitute procedural compliance for outcome measurement, because the outcome metric - user adoption - is structurally unavailable to the evaluators. Model A's evaluation should be dominated by outcomes.

Prediction 3 (Schumpeter). Under Model A, libraries with significant quality defects should be superseded by better alternatives. Under Model B, libraries with significant quality defects should persist indefinitely.

Prediction 4 (Olson). Under Model B, proposals backed by concentrated organizational resources should advance faster than proposals backed by diffuse community effort, independent of technical maturity.

Prediction 5 (Buchanan). Model A's feedback mechanism - users choosing alternatives - should produce faster quality iteration than Model B's feedback mechanism - committee review on multi-year cycles.

Prediction 6 (Stigler). Over time, Model B's output should increasingly reflect the priorities of entities with the most institutional representation, rather than the priorities of the broader user community.

---

5. Observations

This section collects data from the C++ ecosystem and examines each prediction against the public record. The data sources are WG21 papers, committee meeting minutes, Boost mailing list archives, published benchmarks, and public statements by committee participants. Every quotation is attributed with date and source.

5.1 Observation 1

Prediction 1 stated that Model A libraries should more accurately reflect the needs of the broader user community than Model B libraries.

Christopher Kohlhoff is an individual developer with no corporate sponsor. He wrote Boost.Asio and has maintained it for over twenty years. The library has millions of deployed users across every major platform. It formed the basis of the Networking TS[15]. The Networking TS has not been standardized.

The Graph Library (P3126R0)[16] was proposed for standardization in 2024. Section 10 of P3126R0 states: "There is no current use of the library." Section 11 states: "There is no current deployment experience."

The C++ JSON ecosystem contains five competing libraries, each serving a different user need[17]:

Library	Parse Throughput	Niche
simdjson	~1163 MB/s	Maximum throughput (SIMD)
glaze	~1200 MB/s	Compile-time reflection
RapidJSON	~416 MB/s	Low-level control
Boost.JSON	~308 MB/s	Modern, balanced

Library	Parse Throughput	Niche
nlohmann/json	~81 MB/s	Developer ergonomics

No committee designed this ecosystem. Five independent authors identified five different user needs and built five different libraries. Users choose among them based on their own requirements. The distributed knowledge of millions of users produced five specialized solutions. One committee-designed JSON library would serve one of those needs.

5.2 Observation 2

Prediction 1 stated that Model A should aggregate information from a larger population through adoption signals.

Victor Zverovich published the `{fmt}` library on GitHub in December 2012[33]. Over eight years of marketplace competition, the library earned 23,375 stars, 2,852 forks, and 440 contributors. It was adopted by Meta (Folly), among other production codebases. The committee recognized the marketplace's verdict and standardized the design as `std::format` in C++20 (P0645)[34] and `std::print` in C++23 (P2093R14)[35].

The marketplace identified what C++ developers needed. The committee adopted the marketplace's output. This is the founding principle working as designed: existing practice was standardized. The committee did not need to invent a formatting library. The marketplace delivered one, the community validated it through adoption, and the committee consolidated the result.

5.3 Observation 3

Prediction 2 stated that Model B's evaluation process should substitute procedural compliance for outcome measurement.

P2274R0[18] (Aaron Ballman, 2020), a document describing WG21's procedures to WG14 members, states: "WG21 finds implementation experience with a proposal to be incredibly valuable but does not have any requirement on implementation experience to adopt a proposal."

The committee's own documentation confirms the absence of an outcome metric. A library can advance from LEWG to LWG to plenary to the International Standard without a single user outside the proposing organization.

Howard Hinnant, writing on the library reflector in July 2016[19], described the metric that Model B lacks: "I should quit asking: 'Has it been implemented?' The correct question is: What has been the field experience? Is there positive feedback from anyone outside your immediate family or people who could have a perceived conflict of interest (such as employees of your company)?"

Hartmut Kaiser characterized the Boost volunteer review process on the Boost mailing list in 2017[20]: "Having the review process being volunteer-driven guarantees a) a real-world need for the library under review, b) fairness of the decision, c) a high quality of the review, d) direct interest in organizing the review by the review manager."

5.4 Observation 4

Prediction 2 stated that Model A's evaluation should be dominated by outcomes.

N3370[36] (Alisdair Meredith, "Call for Library Proposals," 2012) lists what WG21 asks of library proposers: design decisions, technical specifications, impact on the standard, interaction with other proposals, and proposed wording. A Boost formal review asks different questions: does the library compile on all supported platforms, does it have tests, is the documentation adequate, has anyone used it, would the reviewer use it, and does it meet a real-world need[4].

Evaluated Property	Model A (Boost Review)	Model B (WG21 Process)
Has it been used?	Required	Not required[18]
Do independent users exist?	Expected	Not required[18]
Does it compile everywhere?	Tested on CI	Not required before LEWG
Is the documentation good?	Evaluated by reviewers	Not a formal criterion
Does it meet a real need?	Central question	Implicit in design rationale
Is the wording correct?	Not applicable	Central question

One rubric measures whether the library works for users. The other measures whether the paper is ready for the standard.

5.5 Observation 5

Prediction 3 stated that under Model A, defective libraries should be superseded by better alternatives, and under Model B, defective libraries should persist.

`std::variant` and `boost::variant2`. Howard Hinnant wrote on the library reflector in July 2016, before `std::variant` was standardized[19]: "It is now my understanding that we (the committee) have made significant design changes with respect to all existing variants in the field (most notably boost::variant). We

/think/ these are good changes, and I /hope/ that we are right. We won't /know/ if these are good design changes until we have field experience." He continued: "I won't bother to go down the list of libraries where this strategy has failed us in the past."

`std::variant` was standardized in C++17 with a `valueless_by_exception` state - a condition in which the variant holds no valid value[21]. Peter Dimov wrote `boost::variant2` with a never-empty guarantee using double-buffering[22]. Niall Douglas wrote on the Boost mailing list in 2019[23]: "the whole valueless by exception footgun was 100% avoidable, and Boost.Variant2 immediately eliminated that footgun." The defect persists in `std::variant`. The marketplace produced the fix.

`std::regex` and Boost.Regex. Andrey Semashev wrote on the Boost mailing list in July 2021[24]: "my general impression was that all std::regex implementations were slow compared to Boost.Regex, which seems to be one of the fastest implementations." Phil Endecott reported that a simple pattern match taking two seconds with libstdc++ `std::regex` completed in negligible time with Boost.Regex[24]. `std::regex` remains in the standard, unchanged. Boost.Regex, Boost.Xpressive, CTRE, and RE2 are available as alternatives in the marketplace.

`std::unordered_map` and `boost::unordered_map`. After the Boost 1.80 rewrite by Joaquín M López Muñoz, `boost::unordered_flat_map` outperforms `std::unordered_map` by approximately 3x for string keys and 2.9x for integer keys on GCC 12 (x64)[25]. The standard version cannot be rewritten due to ABI constraints. Boost was free to redesign the data structure from scratch.

`std::error_code` and `boost::error_code`. Niall Douglas wrote on the Boost mailing list in November 2023[23]: "Boost's `shared_ptr` is less footgunny than `std::shared_ptr`. [...] `boost::error_code` over the fundamentally unsafe `std::error_code`." He characterized the pattern: "I think that as WG21's increasing dysfunction at standardising library becomes more obvious to the C++ ecosystem, the need for fixed standard library facilities will grow because the standardised ones will become toxic to use in newly written code."

`std::variant`, `std::regex`, `std::unordered_map`, and `std::error_code` were standardized and remain in the standard. `boost::variant2`, Boost.Regex, `boost::unordered_flat_map`, and `boost::error_code` are available as alternatives in the marketplace.

5.6 Observation 6

Prediction 3 stated that where competition is absent, inferior products persist.

`std::filesystem` was standardized in C++17 based on Boost.Filesystem. The standard version is frozen at the C++17 specification. Boost.Filesystem Version 4 was released afterward with breaking changes to improve the design[37]. Subsequent releases added `fdopendir` / `openat` support for resilience to concurrent filesystem modifications (1.85.0), storage preallocation in `copy_file` to reduce fragmentation on Linux (1.85.0), and continued platform-specific improvements through 1.91.0[37]. The marketplace version iterates. The standard version persists unchanged.

Daniel Lemire documented `std::ranges` performance degradation in October 2025[38]. Trimming whitespace from strings using chained views (`drop_while` , `reverse` , `drop_while` , `reverse`) produced 70 instructions per string on GCC 15 versus 24 for a simple imperative loop. Engineers at a C++ company observed measurable performance degradation after switching to `std::ranges` . The simdjson project limited `std::ranges` support because it caused performance loss[38]. In the marketplace, developers choose the imperative alternative. In the standard, the design persists regardless of the performance evidence.

5.7 Observation 7

Prediction 4 stated that under Model B, proposals backed by concentrated organizational resources should advance faster than proposals backed by diffuse community effort.

Stackless coroutines were proposed by Gor Nishanov at Microsoft. Microsoft provided experimental Visual Studio support in 2013, before formal committee approval. The proposal progressed from initial paper to C++20 standardization in approximately six years[26].

Stackful coroutines were proposed by Oliver Kowalke and Nat Goodspeed, community developers without comparable corporate backing. P0876 reached 19 revisions over twelve years and remains in a "needs-revision" state. Stackful coroutines are not in the C++26 working draft[26].

Both coroutine models are well-understood. Both have multiple implementations. Both serve real use cases. One proposal had Microsoft. The other had two community developers. The timeline difference is twelve years versus six.

5.8 Observation 8

Prediction 4 stated that concentrated organizational resources should predict advancement speed independent of technical maturity.

P2469R0[39] (Kohlhoff, Allsop, Falco, Hodges, Morgenstern, October 2021) states: "The asynchronous model of Asio/Net.TS has evolved to support new use cases while also being careful not to leave existing use cases behind, and the strength of the composition model is testament to that. The model is the result of growth and adaptation from use in the real world, and is one reason it is so widely deployed."

The Networking TS was based on Boost.Asio - the most deployed asynchronous library in C++, with decades of field experience, multiple continuation styles (callbacks, futures, coroutines, fibers, deferred, detached), and production deployment at many companies[39]. `std::execution` (P2300R10)[40] was authored primarily by delegates from NVIDIA, Meta, and other major corporations with significant committee presence. SG4 polled at Kona (November 2023) that networking should use only a sender/receiver model.

P2469R0 observed: "the proposed solution in P2300 forces a single composition mechanism, one for which we have limited field experience, on every user."

The most-deployed async library in C++ was set aside in favor of a framework backed by concentrated institutional resources. Both decisions are in the public record.

5.9 Observation 9

Prediction 5 stated that Model A's feedback mechanism should produce faster quality iteration than Model B's feedback mechanism.

Boost.Serialization was submitted for formal review and rejected. The author, Robert Ramey, revised the library and resubmitted it. It was accepted on the second review. Ramey wrote on the Boost mailing list in January 2023[27]: "The serialization library, after much feedback and consideration was initially rejected by Boost. This was the correct decision as it wasn't really ready by a long shot."

Peter Brett stated in the WG21 admin telecon minutes (N4890, May 2021)[28]: "we dealt with NB comments and we did not do that based on implementation experience. All these bugs that we have found might have surfaced if we had the implementation experience."

Bryce Adelstein Lelbach stated in the same minutes[28]: "the amount of field experience we want to see is very different to how it was 4 or 5 years ago, and I believe that's because we shipped a lot of library features and found a lot of late problems. We have more experience with what happens if we don't make sure that we see field experience."

Arno Schoedl, CTO of think-cell GmbH, wrote on the Boost mailing list on May 9, 2024[29]: "Some recent additions to the standard made questionable design choices, which if a library had been implemented and widely used prior to standardization like in Boost, design choices may have been made differently."

The Boost feedback cycle is: submit, review, reject or accept with conditions, revise, re-review. The cycle time is months. The WG21 feedback cycle is: propose, advance through study groups, vote at plenary, ship in a standard on a three-year cadence. Once shipped, ABI constraints prevent structural revision. Boost.Serialization was rejected, revised, and accepted. `std::variant` was standardized without the field experience Howard Hinnant requested, and the defect persists.

5.10 Observation 10

Prediction 5 stated that Model A's feedback mechanism - users choosing alternatives - should produce faster quality iteration.

`{fmt}` and `std::format` share the same core design and the same original author. The marketplace version has shipped 57 releases since 2012, each incorporating user feedback, performance improvements, and new features[33]. The standard version ships on a three-year cadence and is constrained by ABI stability. The `{fmt}` documentation states it is faster than `(s)printf`, `std::to_chars`, `to_string`, and `std::ostringstream` [33].

Same design. Same author. Different feedback loops. After thirteen years, `{fmt}` is on release 57. `std::format` has been revised in one standard cycle.

5.11 Observation 11

Prediction 6 stated that Model B's output should increasingly reflect the priorities of entities with the most institutional representation.

The committee's own founding document (1990) stated the principle: "The key idea is that the Standard would be based on existing practice"[3]. Joaquín M López Muñoz observed in 2024[6]: "the standards committee has taken on the role of innovator and is pushing the industry rather than adopting external advancements or coexisting with them."

The quality bar between the two models has inverted. The author wrote on the Boost mailing list in December 2023[30]: "As authors have discovered that the bar of quality for standardization is considerably lower than that required for inclusion in the Boost library collection, Boost is no longer seen as a waypoint along the journey to standardization."

Bryce Adelstein Lelbach stated at CppNow 2021[31] that other C++ libraries "can be quickly developed and distributed... the C++ standard library does not have that luxury." The constraint is real. The consequence is that the committee's library output is shaped by what the committee can evaluate - which is bounded by the delegates in the room.

5.12 Observation 12

Prediction 6 stated that over time, Model B's output should increasingly reflect the priorities of entities with the most institutional representation rather than the broader user community.

The following table traces where each major library addition originated.

Standard	Model A Origin (Marketplace)	Model B Origin (Committee)
TR1/C++11	`shared_ptr`, `function`, `bind`, `regex`, `random`, `filesystem`, `chrono`	
C++17	`optional`, `variant`, `any`, `string_view`, `filesystem`	parallel algorithms

Standard	Model A Origin (Marketplace)	Model B Origin (Committee)
C++20	`format` (fmt), `ranges` (range-v3, modified)	concepts, modules, coroutines, `consteval`
C++23	`flat_map` , `stacktrace` , `print` (fmt)	`expected` , `mdspan` , `generator`
C++26		`std::execution` , contracts, reflection

In the TR1 era, nearly every library addition originated from marketplace-tested code. By C++26, the committee's library output is predominantly committee-originated. The founding principle was "existing practice." The table documents the trajectory.

---

6. The Record

Two models of library development are observable in the C++ ecosystem. Section 2 described their properties. Section 3 presented seven findings from the economic literature on centralized allocation and competitive markets. Section 4 derived six predictions from those findings. Section 5 tested each prediction twice - twelve observations drawn from real C++ libraries, committee documents, marketplace data, and the public statements of committee participants.

The predictions and the observations are in the preceding sections. The correspondence between them is the reader's to evaluate.

The record is public.

Acknowledgments

The authors thank Adam Smith, Friedrich Hayek, Ludwig von Mises, Joseph Schumpeter, Mancur Olson, George Stigler, and James Buchanan for the theoretical framework. The authors thank Bjarne Stroustrup for the historical record in the HOPL paper. The authors thank Howard Hinnant for the field experience principle and the `std::variant` assessment. The authors thank Joaquín M López Muñoz for the Bannalia analysis and the Boost.Unordered rewrite. The authors thank Peter Dimov for `boost::variant2` . The authors thank Arno Schoedl for the design quality observation. The authors thank Niall Douglas for the creative destruction characterization. The authors thank Robert Ramey for the Boost.Serialization account. The authors thank Aaron Ballman for

documenting WG21's procedures in P2274R0. The authors thank Peter Brett and Bryce Adelstein Lelbach for the field experience statements in the WG21 admin minutes. The authors thank Hartmut Kaiser for the volunteer review characterization. The authors thank Christopher Kohlhoff for twenty years of Boost.Asio.

References

1. Boost.Beast. Vinnie Falco. HTTP and WebSocket library. https://github.com/boostorg/beast
2. C++ Alliance. https://cppalliance.org/
3. X3J16_90-0052. WG21 founding committee minutes, 1990. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/1990/WG21%201990/X3J16_90-0052%20WG21.pdf
4. Boost. Free peer-reviewed portable C++ source libraries. https://www.boost.org/
5. Bjarne Stroustrup. "Evolving a language in and for the real world: C++ 1991-2006." HOPL III, 2007. https://stroustrup.com/hopl-almost-final.pdf
6. Joaquín M López Muñoz. "WG21, Boost, and the ways of standardization." Bannalia, May 2024. https://bannalia.blogspot.com/2024/05/wg21-boost-and-ways-of-standardization.html
7. Friedrich Hayek. "The Use of Knowledge in Society." ***American Economic Review***, 35(4):519-530, 1945. https://doi.org/10.1142/9789812701275_0025
8. Ludwig von Mises. "Economic Calculation in the Socialist Commonwealth." 1920. https://mises.org/library/economic-calculation-socialist-commonwealth-0
9. Ludwig von Mises. ***Bureaucracy***. Yale University Press, 1944. https://cdn.mises.org/Bureaucracy_3.pdf
10. Joseph Schumpeter. ***Capitalism, Socialism and Democracy***. Harper & Brothers, 1942.
11. Mancur Olson. ***The Logic of Collective Action***. Harvard University Press, 1965.
12. George Stigler. "The Theory of Economic Regulation." ***Bell Journal of Economics***, 2(1):3-21, 1971.
13. James Buchanan and Gordon Tullock. ***The Calculus of Consent***. University of Michigan Press, 1962.
14. Adam Smith. ***The Wealth of Nations***. 1776.
15. Christopher Kohlhoff. Boost.Asio and the Networking TS. https://www.boost.org/doc/libs/release/doc/html/boost_asio.html
16. P3126R0. Phil Ratzloff, Andrew Lumsdaine. "Graph Library: Overview." 2024. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3126r0.pdf

17. JSON library benchmarks. Stephen Berry. https://github.com/stephenberry/json_performance

18. P2274R0. Aaron Ballman. "C and C++ Compatibility Study Group." 2020. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2274r0.pdf

19. Howard Hinnant. Library reflector posts, July 2016. Quoted with permission in P4046R0[32].

20. Hartmut Kaiser. Boost mailing list, March 2017. Subject: "[boost] [review queue] Proposed new policy to enter the review queue."

21. P0308R0. Axel Naumann. "Valueless Variants Considered Harmful." 2016. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0308r0.html

22. Boost.Variant2. Peter Dimov. A never-valueless variant type. https://www.boost.org/libs/variant2/

23. Niall Douglas. Boost mailing list, November 2023. Subject: "[boost] [scope] Scope review starts on November 26th."

24. Andrey Semashev and Phil Endecott. Boost mailing list, July 2021. Subject: "[boost] Regexp performance guarantees."

25. Boost.Unordered benchmarks. Joaquín M López Muñoz. https://github.com/boostorg/boost_unordered_benchmarks

26. Vinnie Falco. "Stackful Coroutines in WG21: A Case Study in Institutional Dynamics." 2025. https://www.vinniefalco.com/p/stackful-coroutines-in-wg21-a-case

27. Robert Ramey. Boost mailing list, January 2023. Subject: "[boost] [Aedis] Formal Review: We could have done better."

28. N4890. Nina Ranns. "WG21 2021-05 Admin telecon minutes." 2021. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/n4890.pdf

29. Arno Schoedl. Boost mailing list, May 9, 2024. Subject: "[boost] What is this 'Beman Project Development'?"

30. Vinnie Falco. Boost mailing list, December 2023. Subject: "[boost] Proposal for Inclusion of Async.MQTT5 Library in Boost."

31. Bryce Adelstein Lelbach. "What Belongs In The C++ Standard Library." C++Now 2021. https://www.youtube.com/watch?v=DhOI3eBMWyo

32. P4046R0. Vinnie Falco. "SAGE: Saving All Gathered Expertise." 2026. https://wg21.link/p4046r0

33. {fmt}. Victor Zverovich. A modern formatting library. https://github.com/fmtlib/fmt

34. P0645R10. Victor Zverovich. "Text Formatting." https://wg21.link/p0645r10

35. P2093R14. Victor Zverovich. "Formatted output." 2022. https://wg21.link/p2093r14

36. N3370. Alisdair Meredith. "Call for Library Proposals." 2012. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2012/n3370.html

37. Boost.Filesystem Release History. Andrey Semashev. https://www.boost.org/doc/libs/1_86_0/libs/filesystem/doc/release_history.html

38. Daniel Lemire. "std::ranges may not deliver the performance that you expect." October 2025. https://lemire.me/blog/2025/10/05/stdranges-may-not-deliver-the-performance-that-you-expect/

39. P2469R0. Christopher Kohlhoff, Jamie Allsop, Vinnie Falco, Richard Hodges, Klemens Morgenstern. "Response to P2464: The Networking TS is baked, P2300 Sender/Receiver is not." 2021. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2469r0.pdf

40. P2300R10. Michał Dominiak, et al. "std::execution." 2024. https://wg21.link/p2300r10